Università di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e Informatica

Advanced Programming Languages

C++: classes

Docente:
Marco Grassia
marco.grassia@unict.it

A.A. 2025/2026

Classes

Advanced Programming Languages: C++

Classes

- Classes are the very **heart of C++** and the major difference w.r.t. C
- A **class** is a **user-defined type** that allows defining objects and groups, enabling **encapsulation**, **abstraction**, and **custom semantics**
  - **State → data members** (fields)
  - **Behavior → member functions** (methods)
- **In C++, they range from** very simple ones (like plain data aggregates, i.e., **C-like structs** without function members or constructors) **to complex ones** that manage resources safely (e.g., RAII)

```
// POD
struct RGB {
   // public by default
   int r, g, b;
};
```

```
class File {
   FILE* f_;
public:
   File(const char* path) {
      f_ = fopen(path, "r");
   }
   ~File() {
      if (f_) fclose(f_);
   }
};
```

Comparison with C

- **classes are not supported**
- **structs are** just containers for data that:
  - Can only contain **data members** (no member functions)
  - Have **no constructors**, **destructors**, or **access control** specifiers
  - **Do not support inheritance or polymorphism**

```
// Struct definition and alias (typedef) [.h]
typedef struct Data { int day, month, year; } Data;

// Procedures [.c]
void data_init(Data &d, int g, int m, int a) {
   d.day = g; d.month = m; d.year = a;
}
void data_add_years(Data &d, int years) {
   d.year += years;
}
// Note the reference (&) to avoid copying
// Note the const to avoid modification
void data_print(const Data &d) {
   printf("%02d/%02d/%d\n", d.day, d.month, d.year);
}
// [main or other callers]
Data k;
data_init(k, 28, 6, 1967); data_print(k);

data_add_years(k, 10); data_print(k);
```

Comparison with C

- Guideline: in **C++**, use struct for **simple data aggregates** (POD-like), and class for **encapsulated types with invariants**
- A **Plain Old Data (POD)** is a struct/class type **that**:
  - **Has no user-defined constructors**, **destructors**, **virtual functions, assignment**
  - Has **only public data members**
  - Has **no base classes or non-POD members**
  - Does **not inherit from any other class**
- PODs behave like C structs: **trivial initialization, standard layout, and binary compatibility with C**

```
// POD
struct RGB {
   // public by default
   int r, g, b;
};
```

C++ classes key features

- They allow **access control**: public, protected, private
- They can define **special member functions**: constructors, destructor, copy/move ops (we'll see them later in the course)
- **Encapsulation**: hide implementation details, expose API
- C++ **classes support**:
  - **Polymorphism** (via inheritance and virtual functions)
  - **Generic programming** (via templates)
  - **Value semantics** (copyable objects)
  - **Resource management** (RAII)

C++ classes: member data

- **Data members** (also called *member variables* or *fields*) are variables declared inside a class or struct
- They define the **state** of an object

```
class Point2D {
private: // this is optional, members are private by default
   double x, y;
}
```

C++ classes: access control

- At first the definition of class resembles the one of a structure… except that **all fields are private by default**
- **Private members** (either data and functions) **cannot be accessed** from outside the class, ensuring encapsulation

```
class Data {
   // Fields (private)
   int day, month, year;
};

// The following won't even compile!
Data d;
d.day = 28; // Direct access to data
d.month = 6;    // Direct access to data
d.year = 1967;  // Direct access to data
```

This code will not compile because it attempts to access private members from outside the class, violating access control rules

C++ classes: encapsulation and access control

- **Encapsulation** is the principle of **hiding internal details of an object and exposing only what is necessary**
- Promotes **modularity** and **maintainability** since the implementation may change in time, but the interface is stable
- Prevents **unauthorized access** to internal state
- Achieved through **access specifiers** and **class design**

Access specifier	Meaning
public	Accessible from outside the class
private	Accessible only within the class [default]
protected	Accessible within the class and derived classes [more in the following lectures]

C++ classes: constructor

- A *constructor* is a **special member function** that is **automatically called when a new object is created**
- It **can take parameters** to initialize the object's state and **does not have a return type**, not even *void*
- **If** an **exception happens** inside, **the object is not created** (**but resources allocated** before **may require handling**)

```
class Data {
   // Fields
private:
   int day, month, year;
public:
   // Constructor
   Data(int d, int m, int y) {
      day = d; month = m; year = y;
   }
};

// Usage
// Assignment initialization
Data d = Data(28, 6, 1967); // Constructor call
// Alternative direct initialization
Data d(28, 6, 1967); // Constructor call
```

Both forms invoke the constructor, but direct initialization (Data d(...)) is generally preferred for efficiency

class vs struct in C++

- **In C++, *class* and *struct* define user-defined types,** have **identical semantics** and **support**:
  - **Data** members
  - Member **functions**
  - **Constructors**/destructors
  - **Inheritance**
  - **Access specifiers** (public, protected, private)
- They **only differ in default access level**:
  - **struct** → members are **public by default**
  - **class** → members are **private by defaut**

```
struct S {
    int x; // public by default
    void f(); // public by default
};

class C {
    int x; // private by default
    void f(); // private by default
};
```

Why structs AND classes?

- **Main reasons: history and C compatibility**
- C had struct long before C++ existed
- C++ preserved struct to remain source-compatible with C code and the C mental model (simple aggregates with public data)
- C++ then added **encapsulation, constructors, destructors, inheritance, and operator overloading to *both* class and struct**, so you can use either for full OO features
- In C++, **class and struct are the *same kind of type* with different default access** and **default inheritance**

Hands dirty:
a minimal Matrix (1)

- Let’s build an example step by step
- **A simple integer Matrix type that groups data and operations**
- We’ll **store**:
  - **rows**, **cols** (dimensions)
  - **data** (contiguous int* buffer of size rows * cols) **in row-major order**
- **Constructors and destructors**
  - Ensure a valid, initialized state after construction
  - Allocate and release the heap buffer safely

C++ classes: member functions

- Let's add some ***member functions*** (aka *methods* in OOP)
- **Data members can be referenced directly by name within member functions**
- Function members **can** always **access other members** of the same class, **even if private**

```
class Data {
   int day, month, year;
   // Member function
   void myPrivateMemberFunction() {}

public:
   // Constructor
   Data(int d, int m, int y) {
      day = d; month = m; year = y;
   }
   // Member functions
   void add_years(int years) {
      year += years;
   }



};
```

Note on terminology

- *"As far as the **C++ standard** is concerned, there is **no such thing as a "method".** This **terminology** is used in other OO languages (e.g., Java) to refer to member functions of a class.*
  *In common usage, you'll find that **most people will use "method" and "function"** more or less **interchangeably**, **although some** people will **restrict use** of "**method" to member functions** (as opposed to "free functions" which aren't members of a class)."*
- "***Method is** just a **generic OO-type term**. Methods **do not exist in C++.** If you open the C++ standard, you won't find any mention of "methods". **C++ has functions, of various flavors**."*

https://stackoverflow.com/questions/8596461/in-c-what-is-the-difference-between-a-method-and-a-function

C++ classes: *this* pointer

- *this* is a **special keyword** available inside **non-static member functions that refers to** the **current object** on which the method is being invoked
- As a **pointer**, it must be used with the -> operator
- It is useful when **name conflicts** arise (e.g., between parameters and data members), or when returning the object itself

```
class Data {
   int day, month, year;
   // Member function
   void myPrivateMemberFunction() {}

public:
   // Constructor
   Data(int d, int m, int y) {
      day = d; month = m; year = y;
   }
   void set_day(int day) {
      if (day < 1 || day > 31) {
         throw std::out_of_range("Invalid day");
      }
      this->day = day; // disambiguation
   }

};
```

Hands dirty:
a minimal Matrix (3)

Add some public methods:

- `int& at(int r, int c);`
- `void fill(int value);`

C++ classes: *const* member functions

- Member functions that **do not modify the object's state** should **be marked as** such using the ***const*** *(function)* **trailing qualifier**
- Inside such functions, this has type const T* (instead of T*)
- Note that this is **different from the *const* type qualifier** that prevents data from being modified

```
class Data {
   int day, month, year;
   // Member function
   void myPrivateMemberFunction() {}

public:
   // Constructor
   Data(int d, int m, int y) {
      day = d; month = m; year = y;
   }
   // Member functions
   void add_years(int years) {
      year += years;
   }
   void print() const {
      printf("%02d/%02d/%d\n", day, month, year);
   }
};
```

C++ classes: usage

- **Like any** other **type**, except for the constructor call
- **Static and dynamic allocations are allowed**

```
// Static (stack) allocation
Data d(28, 6, 1967); // Constructor call

d.print();

d.add_years(10);
d.print();

d.set_day(15);
d.print();

// Dynamic (heap) allocation
Data* d1 = new Data(1, 1, 2000);
d1->add_years(1);
d1->print();
delete d1;  // remember to free the memory

auto d2 = new Data(2, 2, 2020);
d2->print();
delete d2; // remember to free the memory
```

Hands dirty:
a minimal Matrix (3)

Add some *const* member functions:

- size_t rows() const { return rows_; }
- size_t cols() const { return cols_; }
- const int& at(int r, int c) const; *// for const Matrix objects*
- to_string() or non-zero count

C++ class constructors: default member init

- **Default member initializers** (C++11+) **provide** per-member **default values**
- They **can be reassigned in the constructor**
- Use **default member initializers** for **simple, unconditional defaults**

```
class Point {
   int x = 0; // default member initializer
   int y = 0; // default member initializer
   float w = 1.0f; // default member initializer
   char name[20] = "Point"; // default member initializer
public:

   Point(int xVal, int yVal,
            float wVal, const char* nameVal) {
      this->x = xVal;
      this->y = yVal;
      this->w = wVal;
      strcpy(this->name, nameVal);
   }
};
```

C++ class constructors: Member Initializer List (MIL)

- A **member initializer list** is a special **syntax** used **to initialize class members before** the **constructor** body **executes**
- It allows you to **directly invoke the appropriate constructor for each member** or base class
- The initializer list appears **between** the **constructor signature and** its **body**
- Syntax:
  CtorName(parameters) : member1(value1), member2(value2), ... { /* ctor body */ }

```
class Point {
   int x, y;
public:
                    // initializer list
   Point(int xVal, int yVal) : x(xVal), y(yVal) {
      // Constructor body (optional)
      // x and y are already initialized here
   }
};
```

C++ class constructors: Member Initializer List (MIL)

- **Note that members are initialized in the order of declaration**, not in the order of the initializer list
- They **support expressions**
- **Useful for efficiency**, as it avoids default construction followed by assignment
- MIL is **required for *const* and reference members**

```
class Point {
   int x = 0; // default member initializer
   int y = 0; // default member initializer
   float w = 1.0f; // default member initializer
public:

   // initializer list
   Point(int xVal, int yVal, float wVal):
         x(xVal), y(yVal), w(pow(wVal, 2)) {
      if (x < 0 || y < 0) {
         throw std::invalid_argument("Negative coords");
      }
   }
   Point(const Point &other)
         : x(other.x), y(other.y), w(other.w) {}
};
```

C++ class constructors: delegation (C++11 and above)

- A **delegating constructor's MIL has exactly one entry**: a **call to another constructor of the** same **class**
- The **target constructor** initializes members/base; delegator's **body runs after**

```
class Point2D {
   int x, y;
public:
   // Parametrized constructor with
   // member initializer list
   Point2D(int xVal, int yVal) : x(xVal), y(yVal) {}

   // default constructor using delegating constructor
   Point2D() : Point2D(0, 0) {}
};
```

C++ class constructors: const / references and member initializer list

- **const members** cannot be assigned after construction
- **Reference members** must be bound to an existing object at initialization
- These members **must be initialized directly**, before the constructor body runs
- That's why **member initializer lists** are mandatory in these cases

```
class Point {
   int x, y;
   const int z; // const member must be initialized
   int& ref; // reference member must be initialized
public:
                    // initializer list
   Point(int xVal, int yVal) : x(xVal), y(yVal) {
      z = 0; // ✗ error: const member must be
             // initialized in initializer list
      // Constructor body (optional)
      // x and y are already initialized here
   }
};
```

Hands dirty:
a minimal Matrix (5)

- Add a default delegate constructor
- Add a MIL constructor
- Add a *const* constraint on rows and cols

C++ class constructors: overloading

- A class can define **multiple constructors** with different parameter lists (**constructor overloading**)
- The **constructor invoked** depends on the **actual arguments** provided
- We'll come back on the copy constructor (and others) later

```
class C {
private:
   int a;

public:
   C() { a = 0; } // Default constructor
   C(int i) { a = i; } // Conversion constructor
   C(C &c) { a = c.a; } // Copy constructor
   C(int i, int j) { a = i + j; } // Constructor with parameters
};

C c1; // Default constructor
C c2(1); // Conversion constructor
C c3(c2); // Copy constructor
C c4(7, 8); // Constructor with parameters
```

C++ classes: conversion constructor

- A **single-parameter constructor** is often called a **conversion constructor**
- It allows **implicit conversions** from the argument type to the class type
- However, implicit conversions can lead to **unexpected behavior** like precision loss

```
class Number {
   int value;
public:
   Number(int v) : value(v) {}
};

Number n1 = 3.14; // double → int → Number (implicit)
Number n2 = 2.71f; // float → int → Number (implicit)
Number n3 = static_cast<short>(5); // short → int →
Number (implicit)
```

In the example, double, float and short int values are implicitly converted to int

C++ class constructors: *explicit*

- Use the **explicit** specifier to **disable implicit conversions** from the argument type to the class type and avoid subtle bugs
- **Note that it affects only copy-initialization and not direct initialization**
- **Copy and move constructors should not be explicit**, otherwise you lose the possibility for pass-by-value calls
- A **good practice** is to **first declare your converting constructor as explicit**, and **remove** the specifier **only when the implicit conversion is wanted *by design***

```
class Number {
   int value;
public:
   explicit Number(int v) : value(v) {}
};
// ❌ Error: explicit constructor
Number n1 = 3.14;
Number n2 = 4;
Number n3 = {4};

// ✅ allowed , direct and braces
// initialization are fine with
// explicit constructors
Number n4(3.14);
Number n5{4};
```

Hands dirty:
a minimal Matrix (6)

- Add a conversion constructor from a custom Array type (Matrix would have 1 row with N (array.length) entries
- Make it explicit

C++ classes: Destructor

- A **destructor** is a **special member function** that is used **to release resources and perform cleanup**
- **Same syntax as the constructor, preceded by ~** (tilde) (no return type and no parameters)
- **One per class, cannot be overloaded**
- It is **invoked automatically when** a **stack-allocated object goes out of scope or is when a heap-allocated one is explicitly deleted**

```
class Resource {
public:
   Resource() { data = new int[100]; }
   ~Resource() { delete[] data; } // cleanup

private:
   int* data;
};

// Automatic storage duration
{
   Resource r{}; // Calls constructor
} // Calls deconstructor at the end of the scope

// Dynamic storage duration
Resource* r = new Resource(); // Calls constructor
delete r;                     // Calls deconstructor
```

C++ classes: destructor are called automatically when...

1. **a local-object** (stack-allocated) **goes out of scope**
2. ***delete* is used on a pointer** to a **heap-allocated object**
3. **a temporary object is destroyed** (after the expression is fully evaluated)
4. **a base class** or member object **is destroyed**
5. **a container holding objects is destroyed**

```
{ // 1
    File f("example.txt");
    // Use the file...
}// ~File is invoked

// 2
auto* fp = new File("example.txt");
// Use the file...
delete fp; // ~File is invoked

// 3
File("example.txt")
// Temporary object destroyed at the
//  end of the full expression
```

C++ class example

Custom string implementation

```
class string {
    char* _data = nullptr;
    unsigned _capacity{0};
    unsigned _length{0};

public:
    string(): string("") {}
    string(const char* str) {
        println("Constructor invoked");
        this->_data = strdup(str);
        this->_length = strlen(str);
        this->_capacity = this->_length + 1;
    }

    unsigned length() const {
        return this->_length;
    }

    ~string() {
        println("Deconstructor invoked");
        delete[] this->_data;
    }
};
```

Example:

```
{
    // Automatic storage duration
    // Calls constructor
    string s{"hello world"};
    println("{}", s.length());

} // Calls deconstructor at the end of the scope
```

```
Constructor invoked
11
Deconstructor invoked
```

Hands dirty:
a minimal Matrix (7)

- Add a destructor

C++ classes: types of constructors

1. **Default constructor**
   - No parameters (or all parameters have defaults)
   - Generated automatically if no other constructor is declared
2. **Parameterized constructor**
   - Takes arguments to initialize members

```
class Point2D {
   int x, y;
public:
   // default constructor
   Point2D() : x(0), y(0) {}

   // Parametrized constructor with
   // member initializer list
   Point2D(int xVal, int yVal) : x(xVal), y(yVal) {}

   // default constructor using delegating constructor
   // (equivalent to above, but only one can be used)
   Point2D() : Point2D(0, 0) {}
};
```

Initialization semantics: *new A;* vs *new A();* (and {})

Class types

- new A; → **default-initialization** calls A() (if provided)
- new A(); → **value-initialization** calls A() too (for class types, both end up invoking the default constructor if available)
- new A{…} → **list-initialization** uses initializer-list or aggregates' rules

POD / fundamental types (e.g., int or int inside POD)

- new int; → **default-initialization** ⇒ **indeterminate value (even for members of aggregates [PODs])**
- new int(); → **value-initialization** ⇒ **zero-initialized** (*p == 0)
- new int{42}; → initialized to 42

Takeaway: for scalars, **prefer () or {}** to avoid indeterminate values. For user types, both new A; and new A(); call the default ctor, but {} can select list-ctors or prevent narrowing. Fundamental members of aggregates (PODs) are **indeterminate in default init**

C++ classes: types of constructors

3. **Copy Constructor**
   - **Creates a new object as a copy of another**
   - Needed for **pass-by-value** and returning objects
   - **Careful when copying resources (eg. pointers)**: who's the owner after the copy?!
   - **=default behavior**: member-wise copy (this will make sense later…)
   - Can be made private (or, better, **deleted**) to prevent copying

```
class Point2D {
   int x, y;
   int* resource;
public:

   // Copy constructor
   Point2D(const Point2D &other) :
                x(other.x), y(other.y) {
      // Just copying the pointer (shallow copy)
      // Who's responsible for deleting the
      //    resource now?!
      // In this case, both objects will try to delete it,
      // leading to undefined behavior (e.g., double free)
      // In this case, either use deep copy or
      // make the copy-constructor private / deleted
      this->resource = other.resource;
   }
};
```

C++ classes: types of constructors

4. **Move Constructor (C++11+)**
   - Transfers resources from a temporary object
   - Peculiarities: avoids deep copies for expensive resources
   - Often paired with **move assignment,** a special operator override (not sure we'll discuss it)
   - Mark noexcept for performance

```
class Point2D {
   int x, y;
   int* resource;
public:

   // Move constructor
   Point2D(Point2D &&other) noexcept :
            x(other.x), y(other.y) {
      other.x = 0;
      other.y = 0;
      // Steal the resource pointer
      //  and take ownership of it
      this->resource = other.resource;
      // prevent double deletion
      other.resource = nullptr;
   }
};
```

Hands dirty:
a minimal Matrix (8)

- Add a copy and a move constructors

C++ classes: static data members

- **Static members belong to the class** and not to the instance: there is **one per class**, not per object (for templates: one per specialization)
- **Static storage duration**, they exist for the entire program lifetime. They are created before main, destroyed after main, but **the initialization order can cause issues like** Static Initialization Order Fiasco (**SIOF**)
- **Access via** `ClassName::member`. They can also be accessed via an object, but not via *this*
- They're subject to **access control** like any other member (public/protected/private)

```
class Point {
   double x_{0.0}, y_{0.0};
public:
   static unsigned instance_count;

   Point(): Point(0, 0) {}
   Point(double x, double y):
      x_(x), y_(y) {
      instance_count++;
   }
   static Point origin() {
      return Point{0, 0};
   }
   static Point sum(
      const Point &p1,
      const Point &p2) {
      return Point{
         p1.x_ + p2.x_,
         p1.y_ + p2.y_
         };
   }
};
unsigned Point::instance_count = 0;
```

C++ classes: static member functions

- Ideal **for behaviors that conceptually belong to** the **class but don't depend on a specific instance**
- They **cannot be const**, **override**, **or virtual;** they can access **only static** members directly
- They also **have access to** the class's **private members**
- **Overloading is allowed**
- **Access as** T::name

```
class Matrix {
public:
   static Matrix identity(size_t n);
   static Matrix zeros(size_t r, size_t c);
};
```

```
class Angle {
    double radians_{0.0};
public:
    Angle() {};
    explicit Angle(double rad) : radians_(rad) {}

    static Angle Degrees(double deg) {
       return Angle(deg * 3.1415926 / 180.0);
    }
};
```

Hands dirty:
a minimal Matrix (9)

- Static eye member function (return a 2-D array with ones on the diagonal and zeros elsewhere)

C++ classes: self-reference

- A class can contain **members that refer to:**
  - **objects of the same class** (e.g., next/prev pointers in linked lists)
  - **itself** through the implicit this pointer

```
class ListNode {
   // Pointer to the next node in the list
   ListNode *next = nullptr;
   // [Data member missing]

public:
   void link(ListNode &n);
};

// Link node n to this node as its next node
void ListNode::link(ListNode &n) {
   n.next = this;
}
```

C++ classes: self-reference & chaining

- return *this; returns a **reference** to the current object, which enables **fluent APIs through chaining**
- The **return type** of the function must match the fact that you intend to **return a reference**
- **Return by value only when you intend to copy/move**

```
class Date {
   unsigned short day{1}, month{1};
   unsigned year {1970};
public:
   Date& set_day(unsigned short d) {
      day = d;
      return *this;
   }
   Date& set_month(unsigned short m) {
      month = m;
      return *this;
   }
   Date& set_year(unsigned y) {
      year = y;
      return *this;
   }
   Date duplicate() const {
      return (*this); // use copy constructor (redundant)
   }
};
Date d;
d.set_day(15).set_month(8).set_year(1947);
Date d2 = d.duplicate();
```

Hands dirty:
a minimal Matrix (10)

- **Scalar multiplication** operator (**returns a ref** to make it chainable)
- **Copy of the matrix**

Interface and Implementation

In C++, a **class** is **typically split into two parts**:

- **Interface**: **what the class exposes to the outside** world. Usually contained in a .hpp, .h should be used only for headers which can be used from plain C code, and classes are not
- **Implementation**: how the class actually works internally
  - **Member functions** (including the constructor) can be implemented by specifying:

    [return_type>] ClassName::MemberFunction() { */* fn body */* }

  - **Member Initialization Lists goes in the .cpp only**

Interface (.hpp)

```
// MyClass.h

class MyClass {

public:
    MyClass(int value);
    int getValue() const;

private:
    int data;

};
```

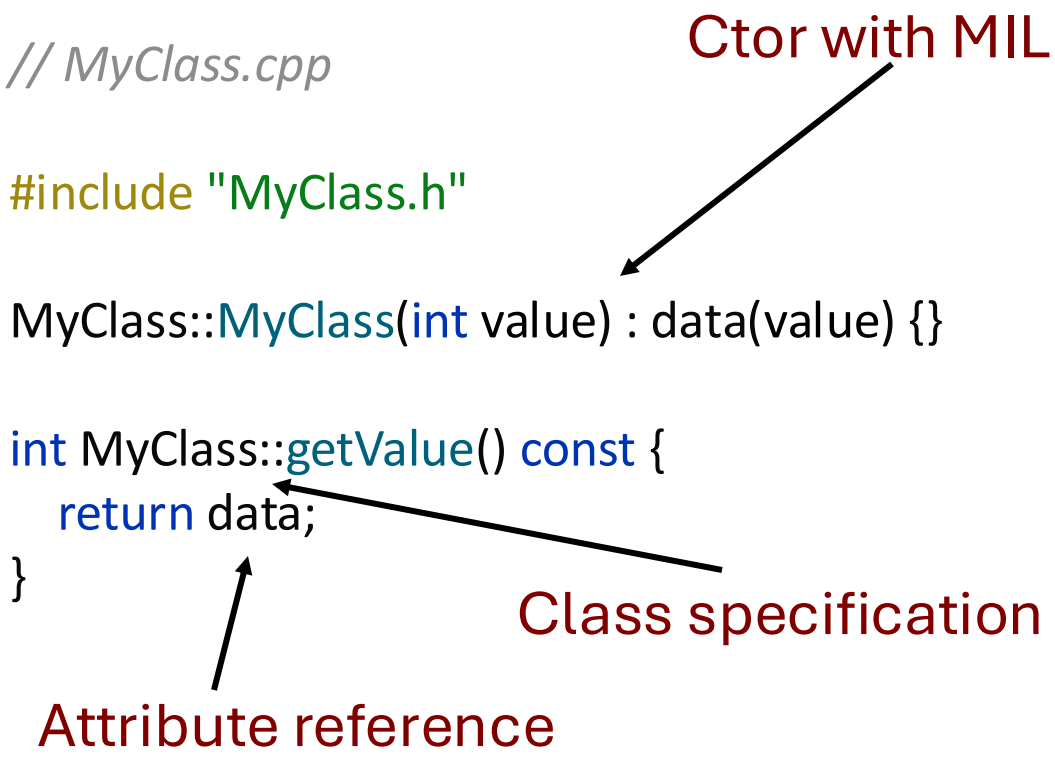
Implementation (.cpp)

```
// MyClass.cpp

#include "MyClass.h"

MyClass::MyClass(int value) : data(value) {}

int MyClass::getValue() const {
    return data;
}
```



Hands dirty:
a minimal Matrix (11)

- Create an interface and implement it

Operator overloading

Operator overloading

- **Operator overloading allows re-defining operator behavior through function definition**
- It is meant to make **user-defined types feel like values, improve readability and composability**: `a + b`, `a == b`, `m[i]`, `p(x)` read naturally
- Enable **idiomatic patterns**: stream I/O (<< / >>), comparisons in algorithms, range-like iteration, fluent transforms
- The **number** of **operands depends on**:
  - The **arity of the operator** (number of accepted arguments)
  - The **operator is a class member or not** (non-member)

What operators can be overloaded

- **Arithmetic**: `+ - * / %`, unary `+ -`, `++ --` (prefix/postfix)
- **Bitwise**: `~ & | ^ << >>`
- **Assignment and compound**: `=`, `+= -= *= /= %=`, `&= |= ^= <<= >>=`
- **Comparison**: `== != < > <= >=` (or single `<=>` in C++20)
- **Logical**: `! && ||` (but **not** `&&`/`||` **short-circuit semantics**)
- **Indexing / call / pointer-like**: [], (), ->, * (dereference)
- **I/O**: << >> (as **non-member** functions; **left operand is a stream**)
- **Memory management**: operator `new`, operator `delete` (and array forms)
- **Conversion operators**: explicit operator T() const

Core constraints and rules you must respect

- **At least one operand must be a user-defined type** (you cannot change built-in types' operator behavior)
- **No new operators**; **precedence**, **associativity**, and **short-circuiting** are **fixed** by the language
- **Don't change fundamental expectations**:
  == must be an **equivalence relation** (reflexive/symmetric/transitive); arithmetic should obey intuitive algebra (or be clearly documented)
- Prefer implementing **compound assignments** (+=, -=…) and **derive** +/- from them for **consistency and efficiency**

What operators cannot be overloaded (by the language)

- . (member access)
- .* (pointer-to-member deref)
- :: (scope)
- ?: (ternary)
- sizeof
- typeid
- alignof, decltype, co_await

Default arguments and MIL for the (explicit) constructor

Operator overloading example

Copy-assignment operator:

- operator= is the function name and returns a Point2D&
- It **takes the source of the data** (*other*) as const ref to avoid it from being copied
- **Does not throw** exceptions
- **Returns a ref to itself** once data is copied

```
class Point2D {
   double x{0.0}, y{0.0};
public:
   // Default (explicit) constructor with default
   // parameters and Member Initializer List syntax
   explicit Point2D(double x = 0.0,
                    double y = 0.0) : x(x), y(y) {}

   // copy assignment operator
   Point2D& operator=(const Point2D &other) noexcept {
      this->x = other.x;
      this->y = other.y;
      return *this;
   }
};
```

Arity & who supplies operands: unary operators

- **Unary** operators (+, -, !, ~, prefix/postfix ++/--)
- As **member,** the sole (implicit) operand is *this
- As **non-member,** the **object is a parameter**
- **Postfix** version **is distinguished by** a **dummy int parameter**
- Remember that some must return copies, others references

```
class Point2D {
   double x{0.0}, y{0.0};
public:
   Point2D operator-() const noexcept {
      Point2D result = *this;
      result.x = -result.x;
      result.y = -result.y;
      return result;
   }
   Point2D& operator++() noexcept { /* Prefix increment */
      this->x++;       this->y++;
      return *this;
   }
   Point2D operator++(int) noexcept { /* Postfix increment */
      Point2D temp = *this;
      ++(*this);
      return temp;
   }
};
inline Point2D operator+(const Point2D& p) {
   return p;
}
```

Arity & who supplies operands: binary op.

- **Binary** operators (+, -, *, /, %, ==, <<, >>, …)
- As **member**: left (implicit!) operand is *this, right operand is parameter
- As **non-member**: both operands are parameters, enabling **implicit conversions on both sides** (often preferred for symmetric ops); could be **marked inline** (we'll discuss about this later)

```
class Point2D {
   double x{0.0}, y{0.0};
public:
   Point2D operator!() const noexcept {
      return Point2D(-x, -y);
   }
   bool operator==(const Point2D& other) const noexcept {
      return (this->x == other.x) && (this->y == other.y);
   }
   bool operator!=(const Point2D& other) const noexcept {
      return !(*this == other);
   }
   bool operator<(const Point2D& other) const noexcept {
      return (std::sqrt(x * x + y * y) <
            std::sqrt(other.x * other.x + other.y * other.y));
   }
   bool operator<=(const Point2D& other) const noexcept {
      return (*this < other) || (*this == other);
   }
}
inline bool operator>(const Point2D& lhs, const Point2D& rhs)
noexcept {
   return rhs < lhs;
}
```

Arity & who supplies operands: binary op.

- **Binary** operators (+, -, *, /, %, ==, <<, >>, …)
- As **member**: left (implicit!) operand is *this, right operand is parameter
- As **non-member**: both operands are parameters, enabling **implicit conversions on both sides** (often preferred for symmetric ops); could be **marked inline** (we'll discuss about this later)

```
class Point2D {
   double x{0.0}, y{0.0};
public:
   Point2D &operator +=(const Point2D &other) noexcept {
      this->x += other.x;
      this->y += other.y;
      return *this;
   }
   Point2D &operator -=(const Point2D &other) noexcept {
      return (*this) += -other;
   }
   Point2D operator+(const Point2D &other) const noexcept {
      Point2D result = *this;
      result += other;
      return result;
   }
}
inline Point2D operator-=(Point2D& lhs, const Point2D& rhs)
noexcept {
   lhs = lhs - rhs;
   return lhs;
}
```

Hands dirty:
a minimal Matrix (12)

- Implement assignation and other operators

Custom conversion operators

- A **conversion operator** is a special member function that converts an object to another type
- It has **no name, no parameters**; **returns implicitly** the **TargetType**
- It **can be const, noexcept, constexpr, ref-qualified** (&, &&)

```
// Syntax
struct T {
   // implicit conversion (unless marked explicit)
   operator TargetType() const;

   // explicit-only conversion
   explicit operator TargetType() const;
};
```

```
class Point2D {
   double x{0.0}, y{0.0};

public:
   explicit operator std::string() const {
      using std::to_string;
      return "Point2D(" + to_string(x) + ", " + to_string(y) + ")";
   }
};
```

Custom conversion operators

- Note that **implicit conversion** operators **participate in overload resolution and can cause ambiguous or unintended matches**
- **Mark** the **operator explicit to require** an **explicit cast** and avoid accidental calls

```
class Point2D {
    double x{0.0}, y{0.0};

public:
    explicit operator std::string() const {
        using std::to_string;
        return "Point2D(" + to_string(x) + ", " + to_string(y) + ")";
    }
};

// Conversion operators in action
// if non-explicit, allows implicit conversion
std::string str1 = p1;
// if explicit, requires static_cast
std::string str2 = static_cast<std::string>(p1);
```

Custom conversion operators: best practices

- **Mark as const if it doesn't mutate**
- For clarity, **provide named functions** to be invoked also by the conversion operator
- **Make string conversions explicit** unless you truly want pervasive, silent conversions

```
class Point2D {
    double x{0.0}, y{0.0};

public:
    std::string to_string() const {
        return "Point2D(" + std::to_string(x) + ", " + std::to_string(y) + ")";
    }
    explicit operator std::string() const {
        return this->to_string();
    }
};
```

Arity & who supplies operands: specials

- operator[] must be a **non-static member, takes one index** (or more with proxy tricks) of type size_t
- **Functor** is a **non-static member** with any parameter list that makes the object **callable. Syntax:**

return_type operator()(<params>)

```
class Point2D {
   double x{0.0}, y{0.0};
public:
   double operator[](size_t index) const {
      if (index == 0) return x;
      else if (index == 1) return y;
      else throw std::out_of_range("Index out of range");
   }
   void operator()(size_t i) const {
      std::cout << "Point2D called as function\n";
   }
};
```

```
class Scaler {
   double factor;
public:
   explicit Scaler() : Scaler(1.0) {}
   explicit Scaler(double factor) : factor(factor) {}

   Point2D operator()(const Point2D &p) const noexcept {
      return Point2D(p[0] * factor, p[1] * factor);
   }
};
```

Clarification on binary logical operators overloading and short-circuiting

- Built-in `lhs && rhs` is *special syntax*: it evaluates `lhs`, and only if `lhs` is false, it **skips** evaluating `rhs`
- **Overloaded** operator**&&** / operator**|| are just functions selected by overload** resolution
- **Function arguments are evaluated first** (before invokation!), so there is **no way to skip** evaluating `rhs` even if `lhs` is false!

Friends

Advanced Programming Languages: C++

C++ classes: *friends*

- *private* and *protected* members of a class cannot be used outside of the class
- A **friend** is a function or class that is **granted access to the private and protected members** of another class
- A **class-friendly** (external) function (or class) **can be declared as such inside the class using the keyword** friend

```
class CSquare {
   int side;
public:
   void set_side(int a) { side = a; }

   friend class CRectangle;
   friend operator<< (std::ostream& os,
                      const CSquare &s) {
      return os << "CSquare(side=" << s.side << ")";
   }
};

class CRectangle {
   int width, height;
public:
   void set_values(int, int);
   int area() { return (width * height); }
   void convert(CSquare a) { width = height = a.side; }
};
```

C++ classes: *friends*

- Friendship is **granted by the class**, not requested by the friend
- Friendship is **not inherited**, **not transitive**, and **not symmetric** (i.e., not mutual):
  - If A declares B as a friend, B can access A's private members
  - But A does **not** automatically access B's private members
  - And if B is a friend of A, C is not automatically a friend of A

```
class CSquare {
   int side;
public:
   void set_side(int a) { side = a; }

   friend class CRectangle;
   friend operator<< (std::ostream& os,
                      const CSquare &s) {
      return os << "CSquare(side=" << s.side << ")";
   }
};

class CRectangle {
   int width, height;
public:
   void set_values(int, int);
   int area() { return (width * height); }
   void convert(CSquare a) { width = height = a.side; }
};
```

C++ classes: *friends*

Use friends when:

- When an external function needs **tight coupling** with a class's internals (e.g., operator overloads like operator<<)
- When another class needs **special access** for efficiency or design reasons
- To implement **non-member functions** that behave like part of the class interface

```
class CSquare {
  int side;
public:
  void set_side(int a) { side = a; }

  friend class CRectangle;
  friend operator<< (std::ostream& os,
                     const CSquare &s) {
    return os << "CSquare(side=" << s.side << ")";
  }
};

class CRectangle {
  int width, height;
public:
  void set_values(int, int);
  int area() { return (width * height); }
  void convert(CSquare a) { width = height = a.side; }
};
```

Hands dirty:
a minimal Matrix (13)

- Implement an operator<< for cout print. Signature:

```
friend std::ostream& operator<<(std::ostream& os, const Matrix &m);
```

Classes: Iterators

Advanced Programming Languages: C++

What are iterators in C++?

- An **iterator** is an **object** that **behaves like a pointer to elements in a container**
- It provides a **standard way to traverse a sequence without exposing** the container's **internal representation**
- They **decouple algorithms from data structures**
- **Instead of** writing a **custom loop for every container**, **you can use generic algorithms** that work with any iterator

C++ iterators: operations an Iterator must support

- **Depending on the category** (input, forward, bidirectional, random-access), **the minimal set varies**. For a simple forward iterator:
- **Dereference:** *it → access the element the iterator points to
- **Increment:** ++it → move to the next element
- **Comparison:** it != end → check if we reached the end
- **Copyability:** you can copy and assign iterators
- **Optional (for richer categories)**
  - --it (bidirectional)
  - it + n (random-access)
  - it - it2 (distance)

C++ iterators: container support

- To make a class iterable, provide begin() and end() methods that return iterators
- For simple containers, these can just return **raw pointers** (which are valid iterators) as they support all required operators (*, ++, !=, ...)

```
class Array {
   size_t size_;
   int* data_;
public:
   explicit Array(size_t n) :
      size_(n), data_(new int[n]{}) {}
   ~Array() { delete[] data_; }

   int& operator[](size_t i) { return data_[i]; }
   const int& operator[](size_t i) const { return data_[i]; }

   // Container support for iteration:
   // They return raw pointers as iterators.
   int* begin() { return data_; }
   int* end()   { return data_ + size_; }

   const int* begin() const { return data_; }
   const int* end()   const { return data_ + size_; }
};
```

C++ iterators: Looping

- When a container provides them, you can iterate using a classic for loop with begin() and end()
- Since C++11, you can write range-based loops that are simpler, safer and less error-prone

```
Array arr(5);

// Basic for loop with iterators
for (auto it = arr.begin(); it != arr.end(); ++it) {
    std::cout << *it << ' ';  // *it dereferences the iterator
}

// Range-based for loop (C++11 and later)
// x is a copy of each element in arr
for (auto x: arr) { // (deref is done automatically)
    std::cout << x << ' ';  // val is the element value
}
```

C++ iterators: Range-based loops

- Since C++11, you can write range-based loops that are simpler, safer and less error-prone
- Internally, it calls begin() and end() for you
- Automatically **dereferences** the iterator for you when needed
- You can control how the values are bound

```
Array arr(5);

// Range-based for loop (C++11 and later)
// x is a copy of each element in arr
for (auto x: arr) { // (deref is done automatically)
   std::cout << x << ' ';  // val is the element value
}

// x is a reference to modify elements
for (auto& x: arr) {
   x += 10;  // modify the element in place
}

// x is const reference: cannot modify elements
for (const auto& x: arr) {
   std::cout << x << ' ';  // val is the element value
}
```

Hands dirty:
a minimal Matrix (14)

1. Add iteration support to the matrix in a element-wise fashion (i.e., from a_{00} to a_{rc})
2. Challenge: usually, an iterator over a Matrix would iterate over the rows (or columns), but it's a bit more complicated
   - Implement a RowIterator class
   - Think of: storing a pointer to the start of the row; knowing the row length (columns); knowing the address of the the first non existent row
   - To further iterate over values in the row, operator* should return another Iterable *provides begin() and end() methods for iteration over the row itself*

Classes: Braces init

Advanced Programming Languages: C++

C++ class constructors: aggregate initialization {values}

- **Aggregate initialization** {<values>} only **applies to aggregates**
- Remember: **aggregates =** no user-declared constructors, no private/protected members, no virtual bases/functions
- Custom classes lose aggregate status if any ctor is declared (C++20 tightened rules)

```
struct Agg { int x; int y; };      // aggregate
Agg a{1, 2};                       // OK: aggregate initialization


struct NonAgg {
   int x, y;
   NonAgg() : NonAgg(0, 0) {}      // delegating ctor
   NonAgg(int a, int b) : x(a), y(b) {}
};
// NonAgg b{1, 2}; // ERROR: no aggregate init anymore
NonAgg b(1, 2); // Must use the constructor
```

C++ class constructors: Initializer list constructor

- std::initializer_list **constructor allows brace-init with** {values} **syntax**
- **IL competes** with other constructors **in overload resolution, but is preferred** when {} is used
- Remember: parens = direct-init; braces = list-init; **braces prevent narrowing** and may prefer initializer_list
- Prefer braces for safety **unless** you specifically need the non-initializer_list overload

```
template<typename T>
struct Vec {
   T* data = nullptr;
   size_t size = 0;

   Vec(std::initializer_list<T> il) {
      size = il.size();
      if (size > 0) {
         data = new T[size];
         size_t index = 0;
         for (int value : il) {
            data[index++] = value;
         }
      }
   }
};

// calls initializer-list constructor
// not aggregate initialization
Vec v{1, 2, 3};
```

Hands dirty:
a minimal Matrix (15)

- Add an initializer list constructor to the matrix:

C++ class constructors: *constexpr* (C++11, expanded in C++14/17)

- *constexpr* **allows object initialization at compile time, enabling** the **type to be used in constant expressions**
- All **members must be initialized with constant expressions**
- Useful for constants, array sizes, template args (e.g., constexpr vars to pass as params)

```
struct Point {
   double x, y;
   constexpr Point(double a, double b):
         x(a), y(b) {}
};

// constexpr initialization: a constexpr variable
// must be initialized at compile time
// Ok, compile-time constants
constexpr Point p1(10, 20);
constexpr Point p2{10, 20};
// NOT ok: x,y not known at compile time
constexpr Point p3(x, y);
constexpr size_t SIZE = 20;
StaticArray<SIZE> a; // OK
```

C++ classes: scoped enums *enum class* (C++11)

- They fix major issues of classic enums
- They are **strongly typed**, **no implicit conversion** to int or other enums
- They are **scoped**: enum **names are not injected into the enclosing scope,** avoiding name clashes
- The **underlying type defaults to int, but can be** explicitly **specified**
- **They allow safer comparisons**: different enum classes cannot be compared accidentally

```
// Basic scoped enum
enum class Direction { North, East,
                       South, West };
Direction d = Direction::North; // ✅
North; // ❌ error: North not in scope
int i = d;   // ❌ error: no implicit conversion
d = 2;     // ❌ error: no implicit conversion
```

```
enum class Color : char {   Red = 'r',
                            Green = 'g',
                            Blue = 'b' };
Color c = Color::Red;
// ✅ explicit conversion
c = static_cast<Color>('g');
```

```
enum class Status : unsigned { Ok = 0,
                               Error = 1 };
Status s = Status::Ok;
s = 1; // Error
```

C++ *enum classes*: using declaration for enums (C++20)

- Scoped enums don't automatically import enumerators
- using can import **enumerators** from a scoped enum into the current scope
- It simplifies access without losing type safety
- Syntax:

  using enum EnumType;

```
enum class Color { Red, Green, Blue };

void paint(Color c) {
   using enum Color;  // imports Red, Green, Blue here
   if (c == Red) { /* ... */ }
}
```

C++ *enum classes*: best practices

- **Always prefer enum class**
- **Avoid conversions** and **explicit underlying type** if possible
- **Avoid ALL_CAPS names**
- Recap: enum class
  - Prevents **namespace pollution** (no global enumerator names)
  - Eliminates **implicit conversions** that cause subtle bugs
  - Improves **type safety** and maintainability

```
// Basic scoped enum
enum class Direction { North, East,
                       South, West };
Direction d = Direction::North; // ✅
North; // ❌ error: North not in scope
int i = d;   // ❌ error: no implicit conversion
d = 2;     // ❌ error: no implicit conversion
```

```
enum class Color : char {   Red = 'r',
                            Green = 'g',
                            Blue = 'b' };
Color c = Color::Red;
// ✅ explicit conversion
c = static_cast<Color>('g');
```

```
enum class Status : unsigned { Ok = 0,
                               Error = 1 };
Status s = Status::Ok;
s = 1; // Error
```

Classes: templates

Advanced Programming Languages: C++

C++ classes: templates

- A template is a **compile-time pattern** that produces concrete declarations only when **instantiated** with arguments (types and/or values)
- Function templates generate functions
- **Class templates generate types**
- A **class template is not a type but a family of types:** you must instantiate it get a real type

```
template<typename T>
class Stack {
   std::vector<T> elems;
public:
   void push(T const&);
   void pop();
   T top() const;
};
```

C++ class templates: kinds of template parameters

- typename (or class) declares a **type parameter.** There is no semantic difference between class and typename in a template-parameter
- **Non-type** parameters (NTTPs): enables compile-time values as parameters
- **Template** template parameters
- ~~**Variadic templates:** parameter packs (typename… Ts) + pack expansion (args…) to handle **0..N** arguments~~
- **Note: default values are supported**

Example: A Stack

```
template<class T>
class Stack {
   T* data{nullptr};
   size_t size_{0};
   size_t capacity_{0};
public:
   explicit Stack(size_t size);
   ~Stack();

   bool push(const T& elem);
   bool pop(T& elem);
   T& top();
   const T& top() const;
   size_t size() const;
   size_t capacity() const;
};
```

C++ templates: non-type template parameters (NTTPs)

- **You can pass** integers / enums / pointers / references / **floating-point / structural literal classes (C++20)**
- This **enables compile-time values as parameters**
- E.g., Array<T, N>

```
template<class T, size_t N = -1, T init = T()>
class Array {
   T* data{nullptr};
   size_t size{0};
public:
   static_assert(N != 0, "Array max size cannot be zero");
   Array(): Array(0) {}
   Array(size_t size) {
      if (size > N)
            throw std::invalid_argument("Size > max array size");
      this->size = size;
      if (size > 0) {
         this->data = new T[size]{};
         for (size_t i = 0; i < size; ++i)
            this->data[i] = init;
      }
   }
};
```

C++ class templates: template alias

- Allows creating **type aliases for templates** using using
- **Template Alias is only possible with using**
- It **simplifies complex template names** and improves readability and maintainability

```
template<typename T>
using IntVec = std::vector<T>;
// Vec<int> instead of std::vector<int>
```

using with templates and dependent bases

- In templates, names in dependent bases are **not found by unqualified lookup**
- using is essential for dependent names in templates
- using (and sometimes this->) makes them visible

```
template<class T>
struct Base {
   using value_type = T;
   void work(T);
   template<class U> void g(U);
};

template<class T>
struct Derived : Base<T> {
   using typename Base<T>::value_type; // dependent type
   using Base<T>::work;              // function
   using Base<T>::g;                 // template function
};
```

Hands dirty: a RingQueue

- Implement a **fixed-capacity FIFO queue** using a **circular buffer** (a raw array T buf[N]; two indices head, tail, plus count)
- **Constraints**: no dynamic allocation; all functions O(1)
- **Hints**:
  - Write a private helper
  - Keep count so you can distinguish empty vs full unambiguously
  - Initialize indices and count in the default constructor

```
template<class T, size_t N, T init = T()>
class RingQueue {
   // returns false if full
   bool push(const T& x);
   // returns false if empty
   bool pop();
   // precondition: not empty
   T& front();
   const T& front() const;
   bool empty() const;
   bool full() const;
   std::size_t size() const;
   std::size_t capacity() const;
   std::size_t next(size_t i) {
      return (i + 1) % N;
   }
};
```